

15.2

Service Decomposition & Bounded Contexts

Domain-Driven Design fundamentals

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

The Core Problem

When splitting a big system into microservices, the hardest question is: WHERE do you draw the lines? Cut in the wrong place and you get a 'distributed monolith' — all the complexity of microservices with none of the benefits. Domain-Driven Design (DDD) gives us the tools to find those lines correctly.

■ **Simple Analogy:** Imagine a hospital. The word 'Patient' means different things to different departments: to Billing it is an invoice account; to Surgery it is a medical record; to Reception it is an appointment. Each department has its own 'Bounded Context' — its own definition of 'Patient'.

Key Terms

Bounded Context	An explicit boundary where a domain model is valid. Inside it, every term has one precise meaning.
Ubiquitous Language	A shared vocabulary used by developers AND domain experts within one bounded context.
Context Map	A diagram showing all bounded contexts and how they relate to each other.
Anti-Corruption Layer (ACL)	A translation layer between two contexts so each speaks its own language internally.
Aggregate	A cluster of domain objects treated as one unit. Only the root is accessed from outside.
Domain Event	A fact that happened in the domain: OrderPlaced, PaymentFailed, UserRegistered.

Diagram — Bounded Contexts for an E-commerce System

Sales Context Customer = prospect Order = quote Product = catalog item	Fulfillment Context Order = shipment unit Customer = delivery address Carrier = FedEx / UPS
Billing Context Customer = payer Order = invoice Payment = transaction	Support Context Customer = case history Order = complaint reference Ticket = issue

Figure 2: The same word 'Customer' has different meanings in each bounded context

Notice how 'Order' means something completely different in each context. If all contexts shared one Order model, every change would break everyone. Bounded contexts keep each model clean and independent.

How to Decompose Services — Step by Step

1. Talk to domain experts — understand the business, not the code.
2. Find natural language boundaries — when experts use different words for the same thing, that's a context boundary.
3. Draw a Context Map — show which contexts exist and how they talk to each other.
4. Decide on integration patterns — Shared Kernel, Customer/Supplier, or Anti-Corruption Layer.
5. Each bounded context becomes one (or a few) microservices.

Context Map Integration Patterns

Pattern	When to Use
Shared Kernel	Two teams share a small common model. Risky — changes affect both.
Customer/Supplier	Downstream (customer) negotiates API with upstream (supplier).
Conformist	Downstream just accepts whatever upstream publishes.
Anti-Corruption Layer	Downstream translates upstream's model into its own — cleanest.
Published Language	An open, well-documented schema all contexts can use (e.g., CloudEvents).

■ **Real-World Example:** Uber Eats: 'Customer' means diner to the Restaurant team, billing account to Payments, and case history to Support. Each team owns its own model. They communicate via Kafka domain events (OrderPlaced), never sharing a database table.